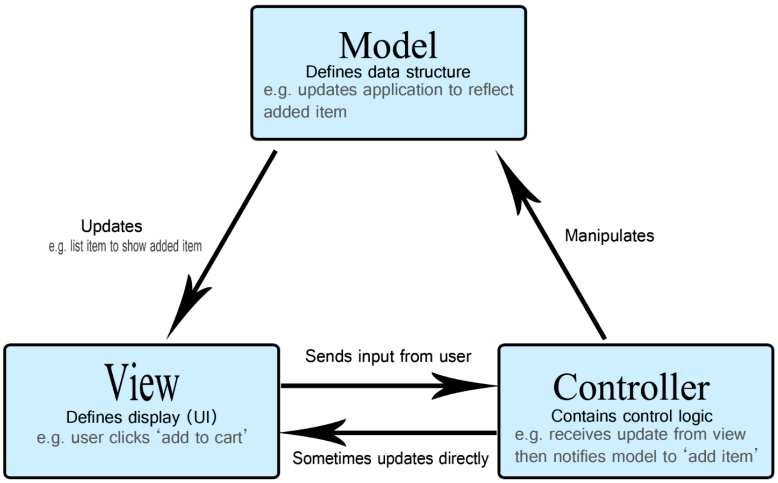
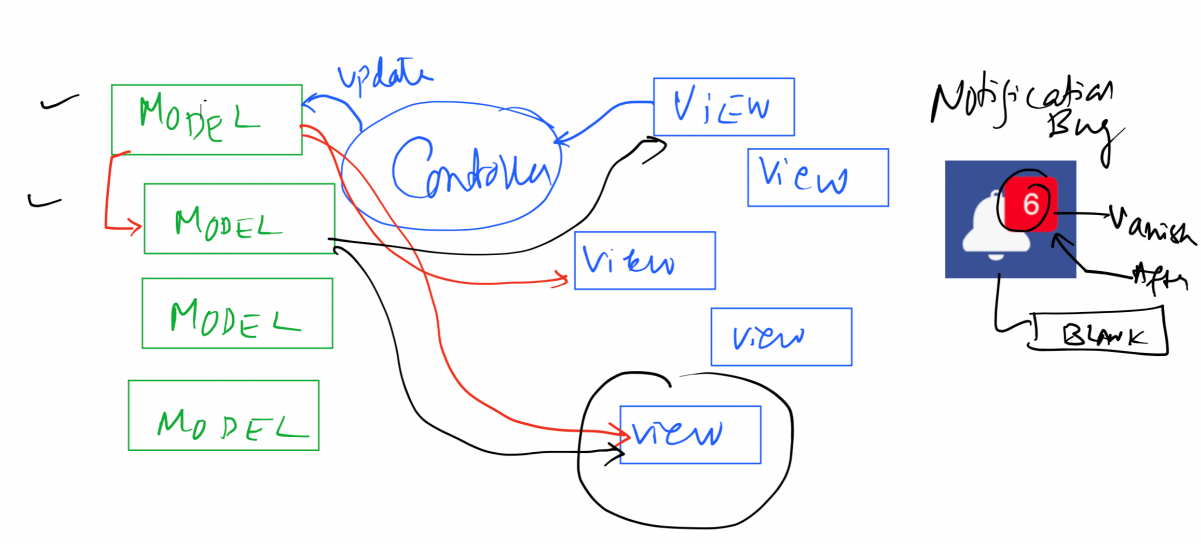
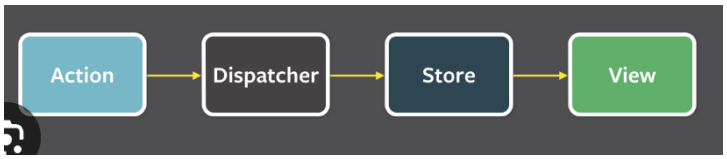


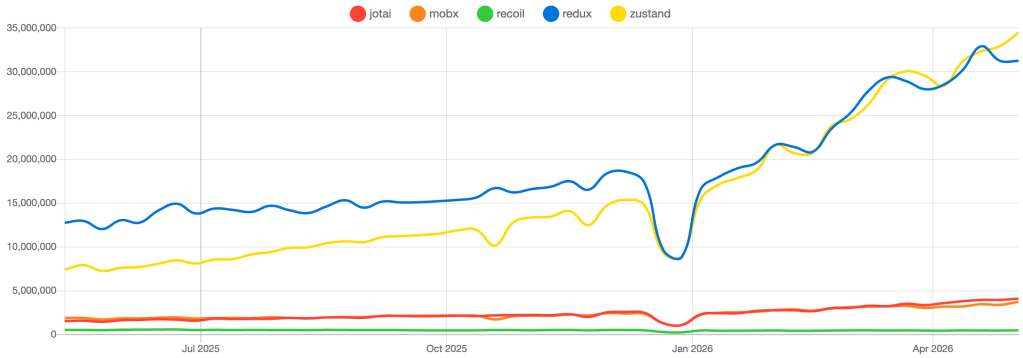

Flux Architecture:
Only way to modify state in store is thro dispatching and action.
Store is an Event Emitter, View was event listener

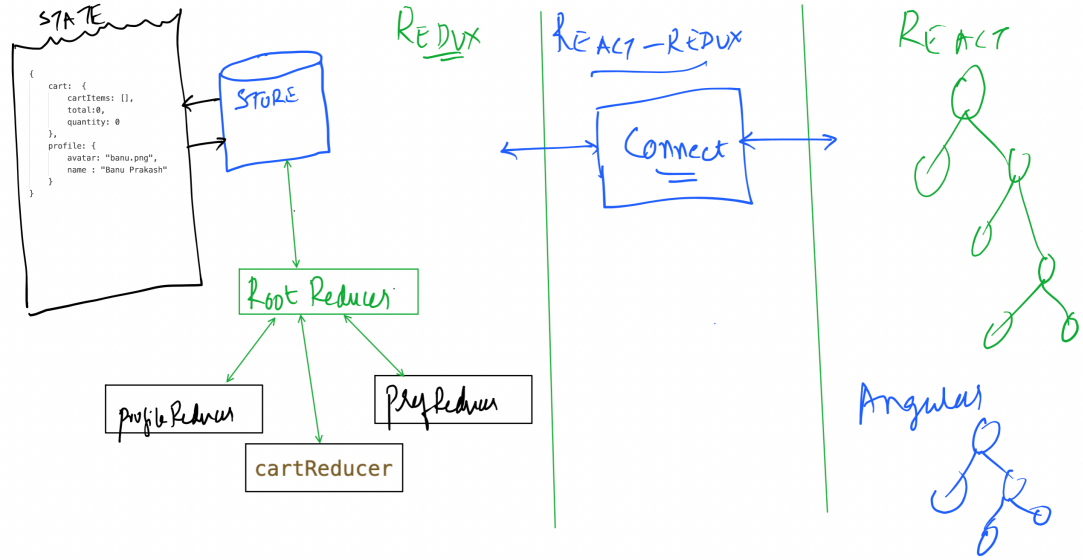


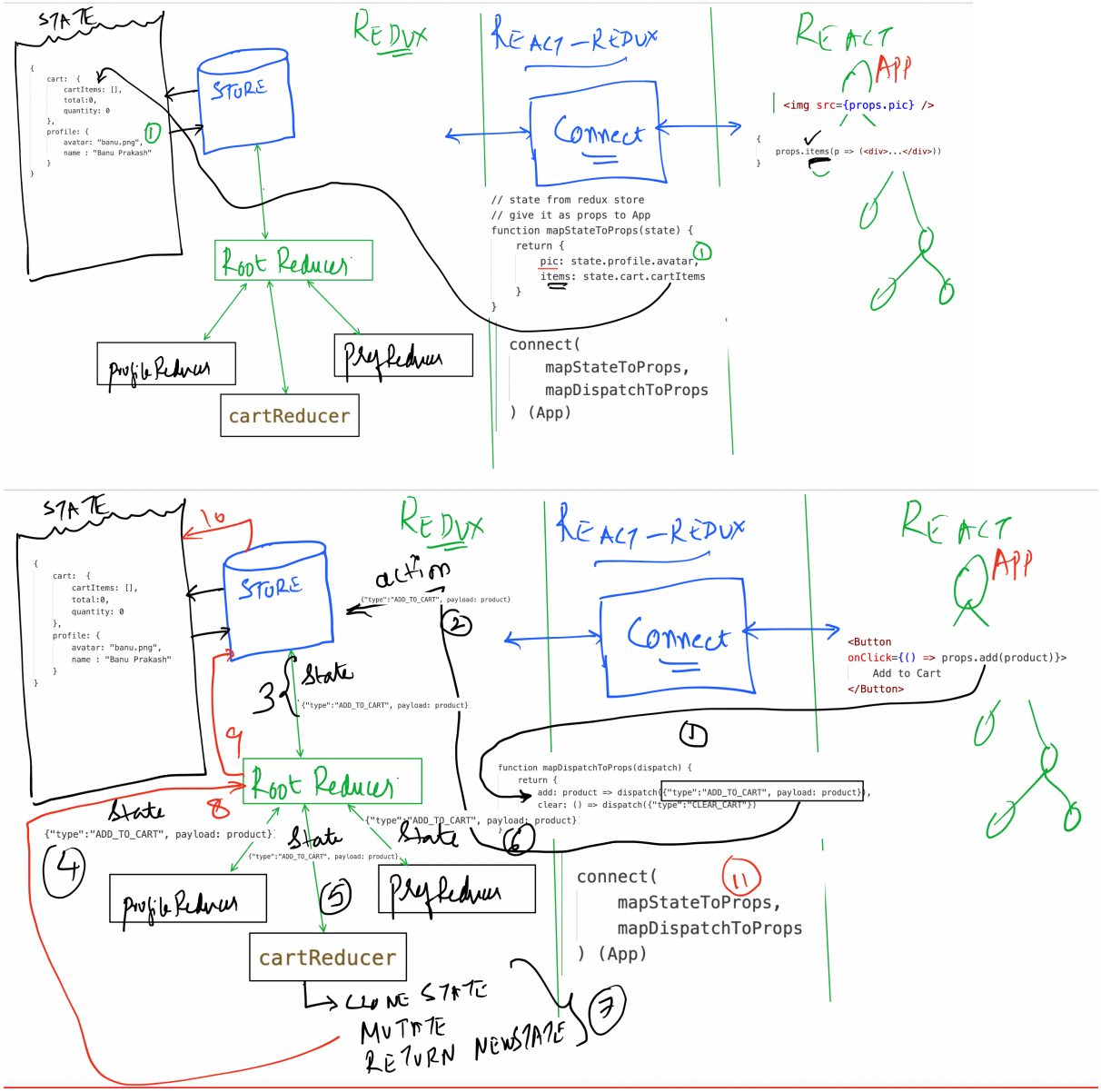

State management libraries for react

jotai × mobx × recoil × redux × zustand ×

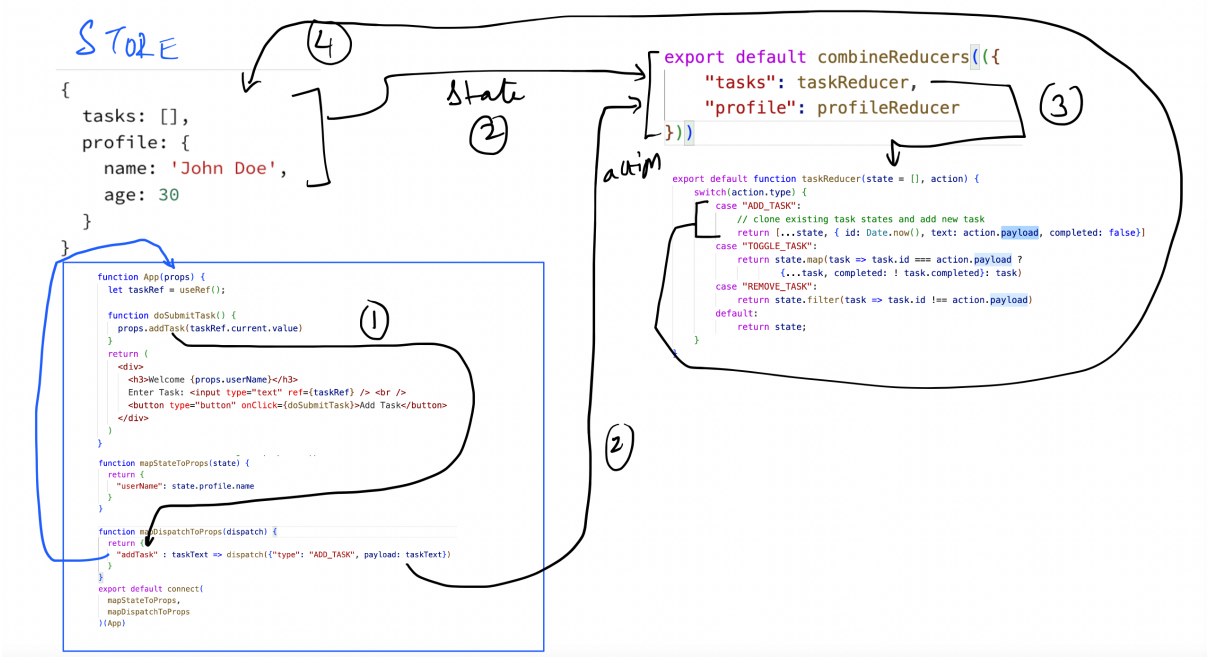
**Downloads** in past 1 Year



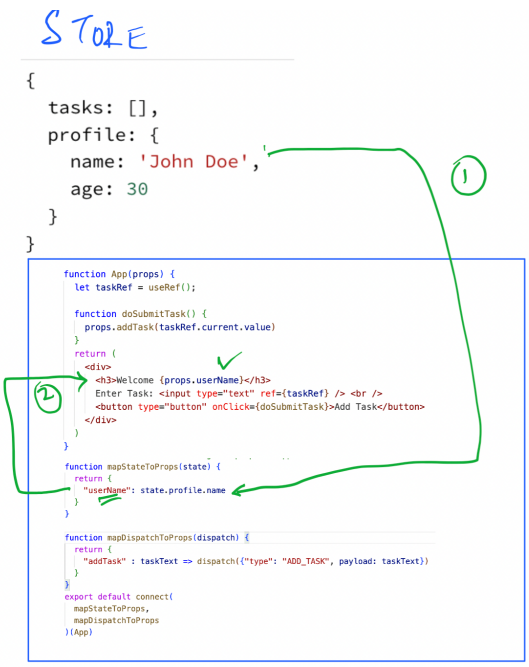

STATE
REDUX
REACT-REDUX
REACT
APP
STORE
Connect
Root Reducer
profileReducer
prefReducer
cartReducer
// state from redux store
// give it as props to App
function mapStateToProps(state) {
return {
pic: state.profile.avatar,
items: state.cart.cartItems
}
}
connect(
mapStateToProps,
mapDispatchToProps
) (App)
<img src={props.pic} />
{
props.items(p => ( ... ))
}
action
State
{"type":"ADD_TO_CART", payload: product}
function mapDispatchToProps(dispatch) {
return {
add: product => dispatch({"type":"ADD_TO_CART", payload: product}),
clear: () => dispatch({"type":"CLEAR_CART"})
}
}
<Button
onClick={() => props.add(product)}>
Add to Cart
</Button>
CLONE STATE
MUTATE
RETURN NEWSTATE

Example:


STORE
{
tasks: [],
profile: {
name: 'John Doe',
age: 30
}
}
state
action
export default combineReducers({
"tasks": taskReducer,
"profile": profileReducer
})
export default function taskReducer(state = [], action) {
switch(action.type) {
case "ADD_TASK":
// clone existing task states and add new task
return [...state, { id: Date.now(), text: action.payload, completed: false}]
case "TOGGLE_TASK":
return state.map(task => task.id === action.payload ?
{...task, completed: ! task.completed}: task)
case "REMOVE_TASK":
return state.filter(task => task.id !== action.payload)
default:
return state;
}
}
function App(props) {
let taskRef = useRef();
function doSubmitTask() {
props.addTask(taskRef.current.value)
}
return (

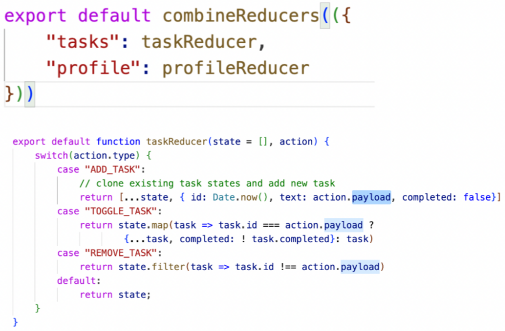
<h3>Welcome {props.userName}</h3>
Enter Task: <input type="text" ref={taskRef} /> <br />
<button type="button" onClick={doSubmitTask}>Add Task</button>

)
}
function mapStateToProps(state) {
return {
"userName": state.profile.name
}
}
function mapDispatchToProps(dispatch) {
return {
"addTask" : taskText => dispatch({"type": "ADD_TASK", payload: taskText})
}
}
export default connect(
mapStateToProps,
mapDispatchToProps
)(App)
1
2
3
4


STORE
{
tasks: [],
profile: {
name: 'John Doe',
age: 30
}
}
function App(props) {
let taskRef = useRef();
function doSubmitTask() {
props.addTask(taskRef.current.value)
}
return (

<h3>Welcome {props.userName}</h3>
Enter Task: <input type="text" ref={taskRef} /> <br />
<button type="button" onClick={doSubmitTask}>Add Task</button>

)
}
function mapStateToProps(state) {
return {
"userName": state.profile.name
}
}
function mapDispatchToProps(dispatch) {
return {
"addTask" : taskText => dispatch({"type": "ADD_TASK", payload: taskText})
}
}
export default connect(
mapStateToProps,
mapDispatchToProps
)(App)
1
2


export default combineReducers({
"tasks": taskReducer,
"profile": profileReducer
})
export default function taskReducer(state = [], action) {
switch(action.type) {
case "ADD_TASK":
// clone existing task states and add new task
return [...state, { id: Date.now(), text: action.payload, completed: false}]
case "TOGGLE_TASK":
return state.map(task => task.id === action.payload ?
{...task, completed: ! task.completed}: task)
case "REMOVE_TASK":
return state.filter(task => task.id !== action.payload)
default:
return state;
}
}